

# assignment #3

## CLAUDE CODE:

```
// CEO Email Prioritizer - object-oriented C++ using a hand-built MaxHeap
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// -----
// Email: one message plus the logic that decides its priority
// -----
class Email {
private:
    string sender;
    string subject;
    string date;      // MM-DD-YYYY
    int    rank;      // higher = read sooner
    long   dateKey;   // YYYYMMDD so newer dates compare larger
    long   sequence; // arrival order, used to break exact ties

    static int rankFor(const string& s) {
        if (s == "Boss")           return 5;
        if (s == "Subordinate")    return 4;
        if (s == "Peer")           return 3;
        if (s == "ImportantPerson") return 2;
    }
};
```

```

        return 1;                                // OtherPerson
    }

    static long keyFor(const string& d) {
        if (d.size() < 10) return 0;
        long mm = stol(d.substr(0, 2));
        long dd = stol(d.substr(3, 2));
        long yy = stol(d.substr(6, 4));
        return yy * 10000 + mm * 100 + dd;
    }

public:
    Email() : rank(0), dateKey(0), sequence(0) {}

    Email(const string& s, const string& subj, const string&
d, long seq)
        : sender(s), subject(subj), date(d),
          rank(rankFor(s)), dateKey(keyFor(d)), sequence(seq)
    {}

    // true if this email should be read before 'other'
    bool higherPriorityThan(const Email& other) const {
        if (rank != other.rank)         return rank > other.rank;

        if (dateKey != other.dateKey) return dateKey > other.dateKey; // newest first
        return sequence < other.sequence;
    } // earlier arrival first

    void display() const {
        cout << "Sender: " << sender << endl;
        cout << "Subject: " << subject << endl;
        cout << "Date: " << date << endl;
    }
};

```

```

// -----
// -----
// MaxHeap: array (list) based, grows dynamically, built from
// scratch
// -----
// -----
class MaxHeap {
private:
    Email* data;
    int    count;
    int    capacity;

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i)   { return 2 * i + 1; }
    static int right(int i)  { return 2 * i + 2; }

    void swapAt(int a, int b) {
        Email tmp = data[a];
        data[a] = data[b];
        data[b] = tmp;
    }

    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];
        for (int i = 0; i < count; i++) bigger[i] = data[i];
        delete[] data;
        data = bigger;
        capacity = newCap;
    }

    void siftUp(int i) {
        while (i > 0 && data[i].higherPriorityThan(data[paren
t(i)])) {
            swapAt(i, parent(i));

```

```

        i = parent(i);
    }
}

void siftDown(int i) {
    while (true) {
        int best = i, l = left(i), r = right(i);
        if (l < count && data[l].higherPriorityThan(data
[best])) best = l;
        if (r < count && data[r].higherPriorityThan(data
[best])) best = r;
        if (best == i) break;
        swapAt(i, best);
        i = best;
    }
}

public:
    MaxHeap() : data(new Email[16]), count(0), capacity(16)
{}

    ~MaxHeap() { delete[] data; }
    MaxHeap(const MaxHeap&) = delete;
    MaxHeap& operator=(const MaxHeap&) = delete;

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }

    void insert(const Email& e) {
        if (count == capacity) grow();
        data[count] = e;
        siftUp(count);
        count++;
    }

    // Returns false if empty; otherwise copies top into 'ou
t' without removing

```

```

bool peek(Email& out) const {
    if (count == 0) return false;
    out = data[0];
    return true;
}

// Removes highest priority email; returns false if empty
bool removeMax() {
    if (count == 0) return false;
    data[0] = data[count - 1];
    count--;
    if (count > 0) siftDown(0);
    return true;
}
};

// -----
// CE0Inbox: wraps the heap and exposes the CEO's commands
// -----
class CE0Inbox {
private:
    MaxHeap heap;
    long    nextSequence;

public:
    CE0Inbox() : nextSequence(0) {}

    void addEmail(const string& sender, const string& subject, const string& date) {
        heap.insert(Email(sender, subject, date, nextSequence++));
    }

    void next() const {

```

```

    Email e;
    if (heap.peek(e)) {
        cout << "Next email:" << endl;
        e.display();
    } else {
        cout << "No emails to read." << endl;
    }
}

void read() { heap.removeMax(); }

void count() const {
    cout << "There are " << heap.size() << " emails to read." << endl;
}
};

// -----
// CommandProcessor: parses the test file and drives the inbox
// -----

class CommandProcessor {
private:
    CEOInbox& inbox;

    static string trim(const string& s) {
        size_t a = s.find_first_not_of(" \t\r\n");
        if (a == string::npos) return "";
        size_t b = s.find_last_not_of(" \t\r\n");
        return s.substr(a, b - a + 1);
    }

    void handleEmail(const string& rest) {
        size_t c1 = rest.find(',');

```

```

        size_t c2 = rest.rfind(',');
        if (c1 == string::npos || c2 == c1) return; // malformed line

        string sender = trim(rest.substr(0, c1));
        string subject = trim(rest.substr(c1 + 1, c2 - c1 - 1));

        string date = trim(rest.substr(c2 + 1));
        inbox.addEmail(sender, subject, date);
    }

public:
    explicit CommandProcessor(CEOInbox& in) : inbox(in) {}

    void process(istream& in) {
        string line;
        while (getline(in, line)) {
            line = trim(line);
            if (line.empty()) continue;
            if (line.compare(0, 6, "EMAIL ") == 0) handleEmail(line.substr(6));
            else if (line == "NEXT") inbox.next();
            else if (line == "READ") inbox.read();
            else if (line == "COUNT") inbox.count();
        }
    }
};

int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        cout << "Enter test file name: ";
        getline(cin, filename);
    }
}

```

```

    ifstream file(filename);
    if (!file) {
        cerr << "Could not open file: " << filename << endl;
        return 1;
    }

    CEOInbox inbox;
    CommandProcessor processor(inbox);
    processor.process(file);
    return 0;
}

```

- Accessed via Google

- **prompt used:**

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

## **Code Analysis:**

### **Correctness**

- The sample test passes and the output matches exactly.
- Weakness: unknown categories fall to the lowest priority, and poor formatted lines are not well accounted for

### **Execution time** (n = number of emails)

- Insert is  $O(\log n)$ , peek is  $O(1)$ , remove is  $O(\log n)$ , and count is  $O(1)$ .
- Array  $O(1)$
- Overall  $O(n \log n)$

### **Space complexity**

- $O(n)$  for the heap array and overall as well/

### **Maintainability**

- division into the four classes
- The heap is reusable and the destructor frees memory.
- Weakness: comments could absolutely be better.

## ChatGPT Code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>

using namespace std;

class Email {
private:
    string sender;
    string subject;
    string date;

    int senderPriority;
    int datePriority;

    int calculateSenderPriority() {
        if (sender == "Boss")
            return 5;
        else if (sender == "Subordinate")
            return 4;
        else if (sender == "Peer")
            return 3;
        else if (sender == "ImportantPerson")
            return 2;
        else
            return 1;
    }

    int calculateDatePriority() {
        int month, day, year;
        char dash1, dash2;
```

```

        stringstream ss(date);
        ss >> month >> dash1 >> day >> dash2 >> year;

        return year * 10000 + month * 100 + day;
    }

public:
    Email(string senderValue, string subjectValue, string dateValue) {
        sender = senderValue;
        subject = subjectValue;
        date = dateValue;

        senderPriority = calculateSenderPriority();
        datePriority = calculateDatePriority();
    }

    string getSender() const {
        return sender;
    }

    string getSubject() const {
        return subject;
    }

    string getDate() const {
        return date;
    }

    int getSenderPriority() const {
        return senderPriority;
    }

    int getDatePriority() const {
        return datePriority;
    }

```

```

bool higherPriorityThan(const Email& other) const {
    if (senderPriority > other.senderPriority)
        return true;

    if (senderPriority < other.senderPriority)
        return false;

    return datePriority > other.datePriority;
}

void display() const {
    cout << "Sender: " << sender << endl;
    cout << "Subject: " << subject << endl;
    cout << "Date: " << date << endl;
}
};

class MaxHeap {
private:
    vector<Email> emails;

    int parent(int index) {
        return (index - 1) / 2;
    }

    int leftChild(int index) {
        return 2 * index + 1;
    }

    int rightChild(int index) {
        return 2 * index + 2;
    }

    void swapEmails(int first, int second) {
        Email temp = emails[first];

```

```

        emails[first] = emails[second];
        emails[second] = temp;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = parent(index);

            if (emails[index].higherPriorityThan(emails[parentIndex])) {
                swapEmails(index, parentIndex);
                index = parentIndex;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        while (true) {
            int left = leftChild(index);
            int right = rightChild(index);
            int highest = index;

            if (left < (int)emails.size() &&
                emails[left].higherPriorityThan(emails[highest])) {
                highest = left;
            }

            if (right < (int)emails.size() &&
                emails[right].higherPriorityThan(emails[highest])) {
                highest = right;
            }
        }
    }

```

```

        if (highest == index)
            break;

        swapEmails(index, highest);
        index = highest;
    }
}

public:
    void insert(const Email& email) {
        emails.push_back(email);
        heapifyUp((int)emails.size() - 1);
    }

    bool isEmpty() const {
        return emails.empty();
    }

    int getCount() const {
        return (int)emails.size();
    }

    void displayNext() const {
        if (emails.empty()) {
            cout << "There are no emails to read." << endl;
            return;
        }

        cout << "Next email:" << endl;
        emails[0].display();
    }

    void removeMax() {
        if (emails.empty())
            return;
    }

```

```

        if (emails.size() == 1) {
            emails.pop_back();
            return;
        }

        emails[0] = emails.back();
        emails.pop_back();

        heapifyDown(0);
    }
};

class EmailManager {
private:
    MaxHeap inbox;

    void processEmail(string line) {
        // Remove "EMAIL "
        string data = line.substr(6);

        size_t firstComma = data.find(',');
        size_t secondComma = data.find(',', firstComma + 1);

        string sender = data.substr(0, firstComma);

        string subject =
            data.substr(firstComma + 1,
                        secondComma - firstComma - 1);

        string date = data.substr(secondComma + 1);

        Email newEmail(sender, subject, date);

        inbox.insert(newEmail);
    }
}

```

```

void processNext() {
    inbox.displayNext();
}

void processRead() {
    inbox.removeMax();
}

void processCount() {
    cout << "There are "
         << inbox.getCount()
         << " emails to read."
         << endl;
}

public:
    void processFile(string filename) {
        ifstream inputFile(filename);

        if (!inputFile) {
            cout << "Unable to open file." << endl;
            return;
        }

        string line;

        while (getline(inputFile, line)) {
            if (line.rfind("EMAIL ", 0) == 0) {
                processEmail(line);
            } else if (line == "NEXT") {
                processNext();
            } else if (line == "READ") {
                processRead();
            } else if (line == "COUNT") {
                processCount();
            }
        }
    }

```

```

    }

    inputFile.close();
}
};

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cout << "Usage: "
              << argv[0]
              << " <test file>"
              << endl;

        return 1;
    }

    EmailManager manager;

    manager.processFile(argv[1]);

    return 0;
}

```

- Accessed via Google
- **prompt used:**

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A

priority queue is a queue where emails can shift towards the front of the queue

based on a priority status.

- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,  
<subject line>,  
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces,

but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

**Correctness:** The code correctly prioritizes emails by sender type and date and handles NEXT, READ, and COUNT correctly.

**Execution time:** Insert and READ take  $O(\log n)$ . NEXT and COUNT take  $O(1)$ .

**Space complexity:**  $O(n)$  because the heap stores all unread emails

**Maintainability:** The code is organized into separate classes with clear jobs, but once again better comments would seem to be helpful/

I am picking Claude because it built the heap with an array and not using vector. I thought some of the edge case handling was also better.

Correctness: Added a quick check to make sure the email category and date look valid before using them

Execution time: Made swapping emails faster by avoiding unnecessary copying

Space: Added a way to reduce the array size when a lot of space is no longer needed

Maintainability: Added clearer comments explaining the classes, priority rules, and important methods.